

JS 版 AIPL ユーザーズガイド

児玉靖司 (Yasushi Kodama)

法政大学 (Hosei University)

2026 年 5 月 12 日

目次

1	概要	2
2	対象実装	2
3	全体構成	2
4	言語構文	3
5	値と式	3
6	アクターランタイム	3
7	組み込み関数	4
7.1	基本	4
7.2	Canvas 描画	4
7.3	デモ可視化用組み込み	4
7.4	ドローンシミュレータ用組み込み	5
8	AI/AIOS/プロトコル機能	5
9	ブラウザ UI	5
10	サンプル	6
11	実行方法	6
11.1	サーバ有り: 通常のブラウザ実行	6
11.2	サーバ無し: Node 実行と静的確認	7
11.3	パーサ再生成	7
11.4	ブラウザ込みスモークテスト	7
12	JS 版の制限	8
13	まとめ	8

1 概要

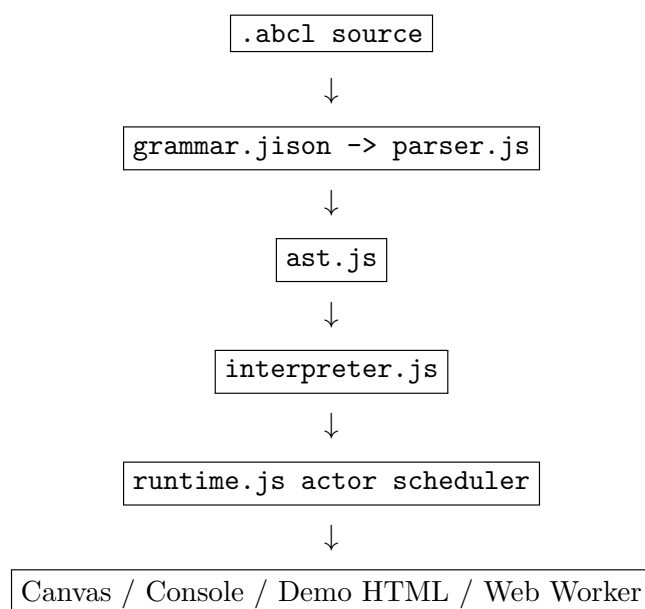
JS 版 AIPL は、src/browser-abcl/ にあるブラウザ向け AIPL 実装である。Jison で生成したパーサ、JavaScript 製 AST、ブラウザ上のアクターランタイム、Canvas 描画、デモ HTML、ブラウザコンソール UI から構成される。

主な目的は、AIPL のアクターモデルをブラウザで可視化しながら実行することである。OCaml 版がネイティブ実行、Python 版が研究用拡張を担うのに対し、JS 版は並行アクター、Canvas 可視化、ブラウザ UI、Web Worker 実験に重点を置く。

2 対象実装

項目	ファイル
パーサ文法	src/browser-abcl/src/parser/grammar.jison
生成済みパーサ	src/browser-abcl/src/parser/parser.js
AST 定義	src/browser-abcl/src/ast.js
インタプリタ	src/browser-abcl/src/interpreter.js
ランタイム本体	src/browser-abcl/src/runtime.js
ブラウザコンソール	src/browser-abcl/src/ui/console_browser.js
Web Worker 実験	src/browser-abcl/src/worker_actor.mjs, worker_runtime.js
デモ入口	src/browser-abcl/index.html

3 全体構成



4 言語構文

JS 版 AIPL は、ブラウザで動かすための中規模な AIPL 実装である。構文は `class`、`method`、`var`、`send`、`now`、`future`、`await`、`select` を中心にしている。

構文	意味
<code>class Name { ... }</code>	クラス定義。
<code>var x = expr;</code>	フィールドまたはローカル変数の宣言。
<code>method name(params) { ... }</code>	アクターが受け取るメソッド定義。
<code>new Class(args)</code>	アクター生成。
<code>send actor.method(args);</code>	非同期メッセージ送信。
<code>send! actor.method(args);</code>	未知メソッド検査を緩める <code>unsafe</code> 送信。
<code>now actor.method(args)</code>	同期呼び出し。相手の <code>reply</code> を待つ。
<code>future actor.method(args)</code>	<code>Future</code> を返す非同期呼び出し。
<code>await futureExpr</code>	<code>Future</code> の結果待ち。
<code>reply(expr);</code>	<code>now/future</code> の呼び出し元へ返信。
<code>if (cond) { ... } else { ... }</code>	条件分岐。
<code>select { case m(args) -> { ... } timeout ms -> { ... } }</code>	メールボックス選択受信。
<code>call builtin(args);</code>	<code>Canvas</code> や <code>AI</code> などの組み込み関数呼び出し。

5 値と式

JS 版 AIPL の値は JavaScript 値として表現される。整数、浮動小数、文字列、真偽値相当の 0/1、アクター名文字列、`Future` オブジェクトを扱う。

式としては、数値・文字列リテラル、変数参照、`new`、関数呼び出し、`now`、`future`、`await`、二項演算を利用できる。演算子は `+`、`-`、`*`、`/`、`==`、`!=`、`<`、`>`、`<=`、`>=` である。

6 アクターランタイム

JS 版のアクターは、JavaScript オブジェクトとして管理される。各アクターは以下の状態を持つ。

- アクター名
- クラス名
- メソッド表
- メールボックス
- インスタンス状態
- 実行中/スケジュール済みフラグ

通常のスケジューリングは `setTimeout` ベースで行われる。`wait(ms)` は現在のアクターの次メッセージ処理

を遅延させる。`now` と `await` は `reply slot` を使い、必要な返信が来るまでメールボックスを同期的に `drain` する。

7 組み込み関数

7.1 基本

関数	概要
<code>print(x)</code>	ログへ出力する。
<code>wait(ms)</code>	アクターの次回処理を遅延させる。
<code>cos(x)</code>	余弦。
<code>sin(x)</code>	正弦。
<code>sqrt(x)</code>	平方根。
<code>abs(x)</code>	絶対値。
<code>floor(x)</code>	切り捨て。
<code>rand(n)</code>	0 以上 <code>n</code> 未満の整数乱数。
<code>randf(n)</code>	0 以上 <code>n</code> 未満の浮動小数乱数。

7.2 Canvas 描画

関数	概要
<code>canvas_line(x1,y1,x2,y2)</code>	Canvas に線分を描く。
<code>sdl_line(x1,y1,x2,y2)</code>	<code>canvas_line</code> と同等の互換名。
<code>canvas_clear()</code> / <code>sdl_clear()</code>	Canvas をクリアする。実装上は再描画時に自動クリアされる。
<code>canvas_present()</code> / <code>sdl_present()</code>	Canvas では即時反映のため <code>no-op</code> 。

7.3 デモ可視化用組み込み

関数	概要
<code>philos_state(id,state)</code>	食事する哲学者の状態を Canvas に反映する。
<code>fork_state(id,state)</code>	フォーク状態を反映する。
<code>fork_free(id)</code>	フォークを空き状態にする。
<code>fork_held(id,holderId)</code>	フォーク保持者を描画する。
<code>buf_init(cap)</code>	bounded buffer 可視化を初期化する。
<code>buf_set(i,v)</code> / <code>buf_clear(i)</code>	buffer slot を更新する。
<code>buf_pstate(s)</code> / <code>buf_cstate(s)</code>	producer/consumer 状態を表示する。

7.4 ドローンシミュレータ用組み込み

関数	概要
<code>world_setup(W,H,comm,view)</code>	ドローン世界を初期化する。
<code>place_obstacle(id,x,y,r)</code>	障害物を配置する。
<code>place_safe(x,y,r)</code>	安全地帯を配置する。
<code>drone_register(id)</code>	ドローンアクターを登録する。
<code>drone_pos(id,x,y)</code>	ドローン位置を更新する。
<code>drone_state(id,state)</code>	ドローン状態を更新する。
<code>drone_scan(id,x,y)</code>	周囲の障害物を検出する。
<code>drone_broadcast(id,obs,x,y,r)</code>	通信範囲内へ障害物情報を拡散する。
<code>drone_remember(id,obs)</code>	障害物を既知情報として保存する。

8 AI/AIOS/プロトコル機能

JS 版 AIPL には、オフラインで動く mock AI 呼び出しが用意されている。

- `ai_call(prompt)`
- `ai_call_with_system(system, prompt)`

標準実装では mock 応答を返す。Node runner 側で `_aiCall` を差し替えることで、外部 AI 呼び出しに接続できる。

AIOS/プロトコル実験用に以下の関数もある。

- `aios_register_service(alias, actor)`
- `aios_now(alias, method, ...)`
- `aios_future(alias, method, ...)`
- `aios_services()`
- `aios_events()`
- `protocol_define(name, spec)`
- `protocol_start(name)`
- `protocol_state(sid)`
- `protocol_end(sid)`
- `protocol_events()`

9 ブラウザ UI

JS 版 AIPL には複数の HTML デモが含まれる。

HTML	内容
<code>index.html</code>	デモ一覧とブラウザコンソールへの入口。
<code>browser_console.html</code>	AIPL ソースをブラウザ上で入力・実行するコンソール。
<code>rotate4lines.html</code>	4 本線の回転 Canvas デモ。
<code>rotate4lines_workers.html</code>	Web Worker を使う回転線デモ。
<code>philosophers.html</code>	食事する哲学者の Canvas 可視化。
<code>bounded_buffer.html</code>	bounded buffer のログ表示版。
<code>bounded_buffer_visual.html</code>	bounded buffer の Canvas 可視化版。
<code>drone_simulator.html</code>	MANET 風ドローン帰還経路シミュレータ。
<code>cooperative_chat.html</code>	Planner/Solver/Reviewer の協調 AI チャット可視化。
<code>cooperative_solve.html</code>	ソース編集、Console、Chat を統合した協調問題解決 UI。

10 サンプル

主な .abcl サンプルは以下である。

- `rotate4lines.abcl`: 4 つの Line アクターが Canvas 上で回転線を描く。
- `philosophers.abcl`: 5 人の哲学者と 5 本のフォークを Canvas で可視化する。
- `bounded_buffer.abcl`: Producer/Consumer と bounded buffer。
- `cooperative_now_future.abcl`: Planner/Solver/Reviewer による `now/future/await` 協調。
- `drone_simulator.abcl`: 災害時ドローンの障害物情報拡散と帰還シミュレーション。

11 実行方法

11.1 サーバ有り: 通常のブラウザ実行

JS 版 AIPL の標準的な実行方法は、`src/browser-abcl/` を HTTP サーバで配信し、ブラウザから `http://localhost:3000/` を開く方式である。この方式では、ES module、絶対パスの `/src/...` 読み込み、Web Worker、HTML デモ間のリンクが自然に動作する。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-claude/src/browser-abcl
npm install
npm run start
```

起動後、ブラウザで次を開く。

```
http://localhost:3000/
```

サーバ有りでの主な利用対象は以下である。

- `index.html`: デモ一覧。
- `browser_console.html`: ブラウザ内 AIPL コンソール。
- `rotate4lines.html`: Canvas 描画デモ。
- `rotate4lines_workers.html`: Web Worker 版デモ。
- `philosophers.html`: 食事する哲学者の可視化。

- `bounded.buffer.html`: bounded buffer ログ版。
- `bounded.buffer.visual.html`: bounded buffer 可視化版。
- `drone.simulator.html`: ドローン帰還経路シミュレータ。
- `cooperative.chat.html`: AI 協調チャット可視化。
- `cooperative.solve.html`: ソース編集付き協調問題解決 UI。

`npm run start` の代わりに、Python の簡易 HTTP サーバでも確認できる。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024/src/browser-abcl
python3 -m http.server 8765
```

この場合は次を開く。

```
http://localhost:8765/
```

11.2 サーバ無し: Node 実行と静的確認

サーバを立てない場合でも、Node.js 上でパーサ、AST、ランタイムの一部を確認できる。これはブラウザ画面を開かずに構文チェックやサンプルのパーズ確認を行う用途に向いている。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024/src/browser-abcl
./_smoke_test.sh
```

`_smoke_test.sh` の通常実行では、主に以下を行う。

- `node --check` による JavaScript 構文確認。
- `parser.js` と `ast.js` を使った `.abcl` サンプルのパーズ確認。

AI 協調サンプルは Node runner でも実行できる。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024/src/browser-abcl
node run_cooperative.mjs
```

なお、HTML を `file://` で直接開くサーバ無し実行は、ページによって制限がある。多くの HTML は `/src/parser/parser.js` や `type="module"` を使うため、ブラウザの CORS 制限、絶対パス解決、Web Worker の制約で動作しないことがある。Canvas だけで完結する一部のスタンドアロン HTML は直接開ける場合があるが、通常はサーバ有り実行を推奨する。

11.3 パーサ再生成

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024/src/browser-abcl
npm run build-parser
```

11.4 ブラウザ込みスモークテスト

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024/src/browser-abcl
./_smoke_test.sh --dynamic
```

このモードでは、内部でローカル HTTP サーバを起動し、Chrome と `puppeteer-core` を使って HTML デモを開く。したがって、これはサーバ有りの動的確認である。

12 JS 版の制限

- Python 版の型注釈、所有権、線形型、transient cast、モデル検査などは JS 版の主対象ではない。
- 通常ランタイムの並行性は `setTimeout` ベースで、厳密な OS スレッドではない。
- Web Worker 版は一部デモ向けの実験実装である。
- 多くのブラウザデモは HTTP サーバ配信を前提にしている。直接 `file://` で開くと、絶対パス、ES module、Web Worker の制約で動作しない場合がある。
- ブラウザ実行のため、ファイル I/O や外部 API 呼び出しは Python 版/OCaml 版ほど直接的ではない。
- AI 呼び出しは標準では mock であり、実 AI 接続は Node runnerなどで `_aiCall` を差し替える。

13 まとめ

JS 版 AIPL は、AIPL のアクター計算をブラウザ上で見える形にするための実装である。言語中核は小さめだが、Canvas 描画、可視化サンプル、ブラウザコンソール、協調 AI デモ、Web Worker 実験を備えている。教育、デモ、並行アクターの挙動確認、可視化プロトタイピングに適したランタイムである。